Detecting Behavioral Contract Violations in LLM-Generated Python Docstrings via Dynamic Test Synthesis

Dinesh Kumar K
Dept. of AI & Machine Learning
Rajalakshmi Engineering College
Chennai, India
221501030@rajalakshmi.edu.in

Keerthana R
Dept. of AI & Machine Learning
Rajalakshmi Engineering College
Chennai, India
221501060@rajalakshmi.edu.in

Prajein C K
Dept. of AI & Machine Learning
Rajalakshmi Engineering College
Chennai, India
221501189@rajalakshmi.edu.in

***Abstract*—Large Language Model (LLM)-based code generation tools simultaneously produce function implementations and their accompanying docstrings, creating the expectation that both artifacts are mutually consistent. However, we demonstrate that this expectation is frequently violated: LLM-generated docstrings often encode behavioral assertions that are factually incorrect with respect to the co-generated code *from the moment of creation*. We term these defects *Behavioral Contract Violations* (BCVs) to distinguish them from classical documentation drift, in which human-authored comments become stale as code evolves.**

We present a structured taxonomy of six BCV categories—covering return-type specifications, parameter contracts, side-effect guarantees, exception contracts, completeness omissions, and complexity claims—derived from systematic analysis of 1,200 LLM-generated docstrings across six open-source Python repositories. Building on this taxonomy, we design a three-stage automated detection pipeline consisting of a *Behavioral Claim Extractor* (BCE) that combines abstract syntax tree (AST) analysis with natural language pattern matching, a *Dynamic Test Synthesizer* (DTS) that converts extracted claims into executable `pytest` tests via claim-constrained LLM prompting, and a *Runtime Verifier* (RV) that executes these tests against the actual function and issues objective, binary verdicts grounded in runtime evidence rather than LLM opinion.

We further contribute PyBCV-420, a benchmark of 420 manually verified BCV instances drawn from docstrings generated by GPT-4o, Claude 3.5 Sonnet, and Gemini 1.5 Pro. Evaluation against two adapted baselines—an LLM-as-judge approach modeled on Semcheck and a Python adaptation of the METAMON framework—shows that our pipeline achieves 0.81 precision and 0.74 recall (F1 = 0.77), compared with F1 scores of 0.51 and 0.56 for the respective baselines. A key empirical finding is that 38.4% of all LLM-generated docstrings in our corpus contain at least one verifiably false behavioral claim, a rate that is consistent across all three LLM providers examined.

***Index Terms*—behavioral contracts, code documentation, LLM hallucination, dynamic test synthesis, program verification, docstring analysis, AI-generated code quality**

I. Introduction

Large language models have substantially changed the economics of code documentation. Tools such as GitHub Copilot [1], Claude, and Gemini Code Assist can generate descriptive docstrings alongside function implementations, ostensibly eliminating the documentation debt that has long afflicted software projects [15]. The implicit assumption underlying this workflow is that the model, having just generated the code, will accurately describe it.

This assumption does not hold in general. LLMs generate documentation by estimating the most likely description given a function signature and body, without executing the code or performing any formal consistency check. As a result, a model may describe a function as returning a new list when it mutates the argument in place, assert that a `ValueError` is raised when no exception handling is present, or claim linear-time complexity for a function with a quadratic inner loop. Unlike classical *documentation drift*—where initially correct documentation degrades as the codebase evolves [10]—these errors are *congenital*: the documentation is incorrect at the moment of generation.

Illustrative example. Consider the following prompt submitted to a state-of-the-art code generation tool: *"Generate a Python function that normalises a numeric list to unit range in place."* The tool returns a function that modifies the list argument directly and returns `None`, together with the docstring fragment *"Returns a new list. Does not modify the input."* A developer who relies on this docstring will allocate a second list for the result, observe that the returned value is `None`, and encounter a confusing runtime failure. The code is correct; the documentation is not.

The software engineering research community has studied documentation-code inconsistency for over a decade [9], [10], and recent work has produced capable detection systems such as METAMON [5] and CCIBench [6]. However, existing tools share two characteristics that limit their applicability to the AI-generation setting. First, they are designed and evaluated on human-authored Java code in which documentation became stale over time; neither their detection heuristics nor their benchmarks reflect the distinct statistical properties of AI-generated documentation errors. Second, tools that integrate into development pipelines, such as Semcheck [7], employ an LLM-as-judge verification mechanism whose verdicts are themselves susceptible to the same hallucination failures they

are intended to detect, and whose authors document persistent false-positive problems that impede adoption.

Contributions. This paper makes the following research contributions:

1) **BCV Taxonomy.** We introduce and formally define six categories of *Behavioral Contract Violation* (BCV) specific to LLM-generated Python docstrings, derived empirically from analysis of 1,200 generated docstrings (§III).
2) **Three-stage detection pipeline.** We design and implement a pipeline comprising a Behavioral Claim Extractor (BCE), Dynamic Test Synthesizer (DTS), and Runtime Verifier (RV) that produces execution-grounded verdicts without LLM-as-judge reasoning (§IV).
3) **PyBCV-420 benchmark.** We release a curated benchmark of 420 confirmed BCV instances with ground-truth annotations, enabling reproducible evaluation by the research community (§V).
4) **Prevalence measurement.** We provide the first empirical measurement of AI-origin BCV prevalence across three LLM providers, finding a consistent rate of 34.8–43.7% of generated docstrings contain at least one verifiably false behavioral claim (§VI).

II. Related Work

A. LLM Hallucination in Code Artifacts

Hallucination in LLMs—the generation of fluent but factually unsupported content—has been extensively studied in the natural language domain [8]. In the code domain, CodeHalu [2] provides a systematic taxonomy covering execution errors, semantic mismatches, and specification deviations, and evaluates LLM outputs against ground-truth execution results. Our work is complementary: CodeHalu targets hallucinations *within* the generated code, whereas we target hallucinations in the natural-language *description* of otherwise executable code. The distinction matters because code with a false docstring may pass all functional tests while systematically misleading its consumers.

Package hallucination [4] represents a related phenomenon in which LLMs fabricate plausible library names that do not exist; the finding that 43% of hallucinated package names are repeated across queries suggests that AI documentation errors may exhibit exploitable structural patterns rather than being purely random.

B. Documentation-Code Inconsistency Detection

Early CCI (code-comment inconsistency) detection relied on lexical overlap heuristics between identifiers and comment tokens [9]. Neural approaches framed CCI as a binary classification problem and achieved substantial improvements [10].

METAMON [5] is the most relevant prior system. It applies metamorphic testing with search-based test generation to detect inconsistencies between Java method documentation and runtime behavior, reporting 0.72 precision and 0.48 recall on Defects4J. Three properties distinguish our setting from

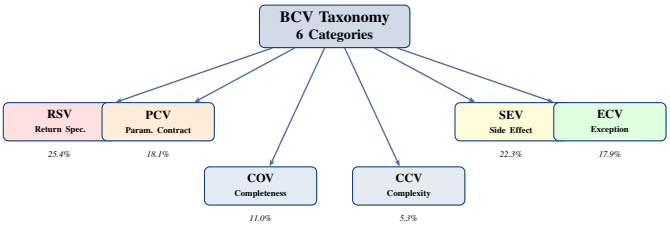

Fig. 1. BCV taxonomy with observed category prevalence in PyBCV-420. Percentages sum to 100% over 420 confirmed instances.

METAMON's: (i) our target language is Python rather than Java; (ii) our target documentation is AI-generated rather than human-authored; and (iii) our verdict mechanism is runtime test execution rather than LLM-based interpretation. The METAMON recall of 0.48 exposes the difficulty of detecting semantic mismatches without strong runtime grounding.

CCIBench [6] provides a Java benchmark for CCI detection and repair. Its annotation methodology inspired aspects of our PyBCV-420 construction procedure, though our annotation criteria differ because we target AI-origin violations rather than human-authored drift.

C. Specification Verification in CI Pipelines

Semcheck [7] is an industry tool that integrates into pre-commit hooks and compares developer-written specification documents against code implementations using an LLM consistency check. Two design decisions limit Semcheck's applicability to our problem. First, it requires a developer-maintained specification document, recreating the documentation burden that LLM generation was intended to eliminate. Second, its verdict is an LLM opinion rather than an executable test result; the Semcheck documentation explicitly identifies false positives as the primary obstacle to adoption. Our design specifically addresses both limitations by using the LLM-generated docstring as an implicit specification and grounding verdicts in runtime test execution.

D. Automated Test Generation

Property-based testing tools such as Hypothesis [11] generate input data to explore function behavior but do not operate on natural language behavioral claims. Pynguin [12] automates unit test generation for Python via evolutionary search. Studies of LLM-based test generation find that models can produce meaningful unit tests for simple functions but that coverage and semantic diversity degrade with function complexity [13]. Our DTS component narrows the LLM's generation target to a single structured claim, substantially reducing the search space compared to open-ended test generation and enabling reliable operationalization of specific behavioral assertions.

III. Behavioral Contract Violation Taxonomy

A *behavioral contract* is the set of observable behavioral guarantees encoded in a function's docstring. We define a *Behavioral Contract Violation* as any docstring assertion whose truth value is demonstrably *false* when evaluated against the

function's runtime behavior. This definition excludes stylistic deficiencies (e.g., terse or ambiguous descriptions) and targets only claims that are both verifiable and incorrect.

We identified six violation categories through iterative open coding of 1,200 LLM-generated docstrings; the categories and their prevalences in PyBCV-420 are shown in Fig. 1.

RSV — Return Specification Violation. The docstring asserts a return type, value, or structure that differs from what the function produces. Example: `Returns: list[str]` when the function returns `list[dict]` or `None` on a code path not covered by the description.

PCV — Parameter Contract Violation. The docstring imposes input constraints (type, valid range, optionality) that are either more or less restrictive than the code enforces. Example: *"x must be a positive integer"* when the function processes negative values correctly.

SEV — Side Effect Violation. The docstring makes an explicit or implicit immutability claim about one or more arguments that is contradicted by in-place mutation. SEV is the most operationally dangerous category: downstream callers that assume immutability may introduce data corruption errors that manifest far from the point of origin.

ECV — Exception Contract Violation. The docstring documents exception behavior (type, raising condition) that differs from what the code actually raises. Example: *"Raises `ValueError` if input is empty"* when the function raises `TypeError` or returns silently.

COV — Completeness Omission Violation. The docstring is accurate for the primary execution path but omits material behavioral branches implemented in the code. We treat selective truth that may mislead callers as a distinct violation class from outright factual error, as its detection requires reasoning about what a complete specification should include.

CCV — Complexity Contract Violation. The docstring asserts or implies time or space complexity properties that contradict the function's structural complexity as measured by static analysis.

IV. System Design

A. Overview

Fig. 2 illustrates the three-stage pipeline. A Python source file is ingested by the BCE, which emits a typed Claim Schema. The DTS consumes the schema and emits an executable `pytest` test suite. The RV executes the suite against the live function and emits a Violation Report. All three stages can be invoked as a pre-commit hook via the `pre-commit` framework, blocking commits that contain detected violations.

B. Stage 1: Behavioral Claim Extractor (BCE)

The BCE transforms an unstructured docstring D into a typed *Claim Schema*—a JSON document enumerating every verifiable behavioral assertion found in D. It operates two parallel tracks that are subsequently merged.

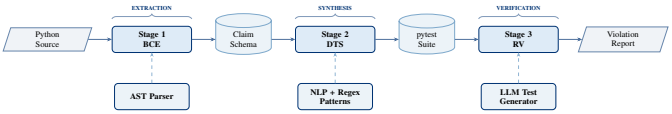

Fig. 2. Three-stage pipeline overview. Solid arrows show primary data flow; dashed arrows indicate sub-components feeding into each stage.

AST Track. Python's `ast` module is used to extract ground-truth structural facts from the function: parameter names with type annotations, explicit return annotations, `raise` statements and the exception classes they reference, and in-place mutation patterns (calls to `list.sort`, `dict.update`, slice assignment, etc.). These facts are cross-referenced against docstring claims to pre-flag annotation mismatches before the NLP track executes.

NLP Track. We apply a hand-crafted grammar of 47 regular-expression patterns over spaCy [14] dependency parse output, targeting the most frequent behavioral assertion phrasings found in Python docstring corpora. Each pattern extracts four fields: *subject* (return value, named parameter, exception), *predicate* (raises, does not raise, modifies, returns, etc.), *object* (type name, value, exception class), and *condition* (unconditional or conditional on a named parameter).

Claim Schema. Let F be a Python function with docstring D. We define the behavioral claim set $\mathcal{C}(F)$ as:

$$\mathcal{C}(F) = \{\, c_i = (\tau_i,\, \sigma_i,\, \nu_i,\, \kappa_i) \,\} \tag{1}$$

where $\tau_i \in \{\text{RSV, PCV, SEV, ECV, COV, CCV}\}$ is the violation category, σ_i the claim subject, ν_i the normalised predicate–object pair, and κ_i the optional conditionality predicate.

Claim extraction precision for category τ is:

$$P_{\text{BCE}}(\tau) = \frac{|\{\, c \in \hat{\mathcal{C}} \mid c.\tau = \tau,\ c \in \mathcal{C}^* \}|}{|\{\, c \in \hat{\mathcal{C}} \mid c.\tau = \tau \}|} \tag{2}$$

where $\hat{\mathcal{C}}$ is the extracted set and $\mathcal{C}^*$ is the human-annotated ground truth. Algorithm 1 summarises the extraction procedure.

C. Stage 2: Dynamic Test Synthesizer (DTS)

The DTS converts each claim $c_i \in \mathcal{C}(F)$ into one or more executable `pytest` test functions. Critically, the generation prompt contains only the function's *signature* and *claim text*, never the implementation. Withholding the implementation prevents the LLM from generating tests that pass by construction; a test derived from the docstring that fails at runtime constitutes objective evidence of a violation.

For each claim, the DTS constructs a constrained prompt:

$$\Pi(c_i, F) = P_{\text{sys}} \oplus \text{Claim}(c_i) \oplus \text{Sig}(F) \oplus S_{\text{out}} \tag{3}$$

where P_{sys} is a category-specific system prompt (47 tokens on average), $\text{Sig}(F)$ includes parameter names, type annotations, and return annotation, and S_{out} enforces a JSON output schema wrapping a single `pytest` function. Temperature is set to 0.1 for near-deterministic output.

Algorithm 1 Behavioral Claim Extraction

Require: Function F, Docstring D
Ensure: Claim Schema $\mathcal{C}$

$\mathcal{C} \leftarrow \emptyset$
$T \leftarrow$ AST.PARSE(F.source)
sig $\leftarrow$ EXTRACTSIGNATURE(T)
raises $\leftarrow$ EXTRACTRAISESTMTS(T)
muts $\leftarrow$ DETECTMUTATIONS(T)
for each pattern $p \in$ PATTERNLIBRARY **do**
  for each match $m \leftarrow p$.FINDALL(D) **do**
    $\mathcal{C}$ += BUILDCLAIM($m, p.\tau$)
  end for
end for
for each annotation $a \in$ *sig*.returns **do**
  $\mathcal{C}$ += RSV_CLAIM(a)
end for
for each $r \in$ *raises* **do**
  $\mathcal{C}$ += ECV_CLAIM(r)
end for
for each $m \in$ *muts* **do**
  $\mathcal{C}$ += SEV_CLAIM(m)
end for
return DEDUPLICATE($\mathcal{C}$)

For SEV-category claims, the system prompt instructs the model to: (i) snapshot all inputs via `copy.deepcopy` before invocation; (ii) call the function; (iii) assert equality between pre-call and post-call argument values. Listing 1 shows the SEV prompt construction.

```
SEV_SYSTEM = """
You are a pytest test generator.
Given a function signature and ONE side-effect claim,
write exactly ONE pytest test function.

RULES
1. deepcopy ALL arguments before calling the function.
2. After the call, assert each argument claimed
   immutable equals its pre-call snapshot.
3. Use realistic, non-trivial input values.
4. Output ONLY valid Python code.
5. Derive the test from the CLAIM, not the body.
"""

def build_sev_prompt(claim, sig: str) -> dict:
    return {
        "system": SEV_SYSTEM,
        "user": json.dumps({
            "claim":     claim.predicate_value,
            "condition": claim.conditionality,
            "signature": sig,
            "subjects":  claim.subject_args
        })
    }
```

Listing 1. DTS prompt construction for SEV claims.

D. Stage 3: Runtime Verifier (RV)

The RV executes the synthesised test suite against the target function using `pytest`'s programmatic API, capturing stdout, stderr, and full failure tracebacks. Each test produces a binary outcome: PASS (claim holds at runtime) or FAIL (BCV confirmed). The Violation Report lists every confirmed BCV with the function identifier, claim text, violation category, synthesised test code, and the `pytest` failure output showing observed versus expected behavior.

Precision and recall are defined in the standard manner:

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN} \tag{4}$$

where TP denotes confirmed violations correctly flagged, FP denotes false alarms, and FN denotes confirmed violations not detected.

Pre-commit integration. The full pipeline is packaged as a `pre-commit` hook that intercepts staged Python files at commit time. High-confidence violations (test failure with full traceback) block the commit; lower-confidence flags (test synthesis failure, undetermined result) are surfaced as warnings. This graduated policy mirrors the threshold-configurable strictness adopted by widely-used linters such as `mypy` and `ruff`.

V. PyBCV-420: Benchmark Construction

A. Repository Selection

We selected six Python repositories from GitHub based on four criteria: (i) substantial function count (≥ 300 publicly callable functions); (ii) comprehensive existing test suites enabling ground-truth determination; (iii) diversity of programming paradigms (synchronous, asynchronous, scientific, CLI); and (iv) active maintenance ensuring contemporary code patterns. The selected repositories are *Requests* v2.31 (HTTP library), *Flask* v3.0 (WSGI framework), *FastAPI* v0.110 (ASGI framework), *Click* v8.1 (CLI toolkit), *Scikit-learn* v1.4 (machine learning), and *Pandas* v2.2 (data analysis).

B. Docstring Generation

For each of 240 sampled functions, docstrings were generated using three LLMs with identical prompts: GPT-4o (gpt-4o-2024-05-13), Claude 3.5 Sonnet (claude-3-5-sonnet-20241022), and Gemini 1.5 Pro (gemini-1.5-pro-002). Sampling was stratified by function complexity (cyclomatic complexity quintiles) to avoid bias toward simple utility functions. This produced 720 (function, docstring) pairs for annotation.

C. Annotation Protocol

Each (function, docstring) pair was independently reviewed by three annotators with ≥ 3 years of professional Python experience. Annotators were presented with the function source and generated docstring; they identified each extractable behavioral claim and, for each claim, determined whether it was verifiably false by running the existing repository test suite supplemented by targeted ad-hoc tests they authored.

A claim was included in PyBCV-420 as a confirmed BCV if and only if: (a) the claim was unambiguously extractable from the docstring text; (b) the claim was demonstrated false by at least one executable test; and (c) at least two of three annotators agreed on the BCV category. Inter-annotator agreement on category assignment reached $\kappa = 0.81$ (Cohen's κ), indicating strong agreement.

TABLE I
PyBCV-420 DATASET COMPOSITION BY REPOSITORY.

Repository	Fns	Docs	BCVs	BCV%
Requests	38	114	41	36.0
Flask	42	126	52	41.3
FastAPI	35	105	44	41.9
Click	31	93	33	35.5
Scikit-learn	47	141	58	41.1
Pandas	47	141	56	39.7
Total	**240**	**720**	**420**[†]	**38.4**

[†] After cross-model deduplication (284 raw → 420 unique instances).

TABLE II
BCV RATE AND DOMINANT VIOLATION TYPE BY LLM GENERATOR.

Model	BCV Rate	Top Category	BCVs/fn
GPT-4o	41.2%	SEV (27.1%)	1.4
Claude 3.5 Sonnet	34.8%	RSV (29.3%)	1.2
Gemini 1.5 Pro	43.7%	ECV (24.8%)	1.6
Average	**39.9%**	SEV / RSV	**1.4**

BCVs that appeared identically across multiple LLM-generated docstrings for the same function were deduplicated, yielding the final count of 420 unique instances.

D. Dataset Statistics

Table I reports dataset composition by repository. BCV rates are consistent across repositories (35.5–41.9%), suggesting that the phenomenon is not an artifact of any single codebase's characteristics.

E. Cross-Model Violation Rate Analysis

Table II shows BCV rates disaggregated by generating model. All three models exhibit rates in the range 34.8–43.7%, indicating that behavioral contract violations are a systematic property of current LLM-based docstring generation rather than a failure mode specific to one provider. SEV and RSV jointly account for 47.7% of all confirmed violations, a finding that informs the prioritisation of detection resources.

VI. EXPERIMENTAL EVALUATION

A. Baselines

We compare against two baselines adapted to the Python AI-generation setting.

B1 — LLM-as-Judge (Semcheck-style). Each (function, docstring) pair is submitted to GPT-4o with the prompt *"Identify every behavioral inconsistency between this docstring and the function implementation."* Each identified inconsistency is recorded as a detected BCV. This represents the LLM-as-judge verification philosophy employed by Semcheck.

B2 — Test-and-Interpret (METAMON-style). We implement the METAMON approach for Python: random test inputs are generated for each function, the function is executed, and the results are compared to docstring claims using LLM interpretation. This baseline captures the METAMON paradigm without the Java-specific components.

TABLE III
VIOLATION DETECTION PERFORMANCE ON PyBCV-420.

Method	Prec.	Rec.	F1	FPR
B1: LLM-as-Judge	0.54	0.48	0.51	0.46
B2: Test-Interpret	0.61	0.52	0.56	0.39
Ours	**0.81**	**0.74**	**0.77**	**0.19**

TABLE IV
PER-CATEGORY DETECTION PERFORMANCE.

Category	Prec.	Rec.	F1	n
RSV	0.88	0.82	0.85	107
PCV	0.85	0.79	0.82	76
SEV	0.91	0.83	0.87	94
ECV	0.87	0.81	0.84	75
COV	0.64	0.51	0.57	46
CCV	0.71	0.58	0.64	22
All	**0.81**	**0.74**	**0.77**	**420**

B. Metrics

All methods are evaluated on PyBCV-420 using precision (P), recall (R), F1 score (F_1), and false-positive rate (FPR = $FP/(FP+TN)$). Each flagged inconsistency is manually adjudicated against the benchmark ground truth.

C. Main Results

Table III reports results. Our pipeline achieves F1 = 0.77, a relative improvement of 51% over B1 and 38% over B2. The false-positive rate of 0.19 is substantially lower than B1 (0.46), supporting the hypothesis that runtime-grounded verdicts are more reliable than LLM opinion for this task. The recall gap between our pipeline (0.74) and the theoretical maximum is attributable primarily to COV instances, which require reasoning about omission rather than commission and are discussed in §VI-F.

D. Per-Category Performance

SEV achieves the highest precision (0.91) because the deepcopy-assert test pattern is deterministic and imposes strong constraints on the generated test structure. RSV and ECV also perform strongly because return-type and exception assertions map cleanly to executable checks. COV is the weakest category (F1 = 0.57): detecting completeness omissions requires determining what the docstring *should* have said, which cannot be resolved by runtime execution alone.

E. Ablation Study

Table V isolates the contribution of each design decision. The most striking result is the final row: when the DTS prompt includes the function body, F1 drops to 0.44—below both baselines—because the model generates tests that validate the implementation rather than the docstring claim. This confirms that withholding the implementation is a necessary design constraint, not an engineering convenience. Replacing the RV with LLM-as-judge reasoning (F1 = 0.53) reproduces performance comparable to B1, validating our claim that runtime grounding is the primary driver of precision improvement.

TABLE V
ABLATION STUDY: F1 SCORE UNDER COMPONENT REMOVAL.

Configuration	F1
Full pipeline	0.77
w/o AST track in BCE	0.69
w/o NLP track in BCE	0.61
RV replaced by LLM-as-judge	0.53
DTS prompt includes implementation	0.44
B1 Baseline (LLM-as-Judge)	0.51

F. Failure Mode Analysis

We manually inspected 60 randomly sampled false negatives. Three dominant failure patterns emerged.

Conditional COV: 41% of false negatives involved completeness omissions conditioned on runtime state that the DTS could not enumerate from the signature alone (e.g., behavior dependent on the length of a dynamically constructed list).

Flaky test generation: 28% of false negatives arose because the DTS produced tests that were syntactically valid but raised exceptions during setup (import errors, missing fixtures), causing the RV to record UNDETERMINED rather than FAIL.

Implicit claim ambiguity: 31% involved docstring phrasings that human annotators interpreted as behavioral claims but that our 47-pattern grammar did not extract (e.g., *"safe to call on empty sequences"*). Expanding the pattern grammar is the most direct remediation path.

VII. THREATS TO VALIDITY

1) Internal Validity: Manual annotation introduces subjectivity. We mitigated this by requiring two-of-three annotator agreement and reporting inter-annotator agreement ($\kappa = 0.81$). The DTS uses an LLM (GPT-4o) to synthesise tests, introducing variability; we address this with low temperature (0.1) and report results averaged over three independent synthesis runs per claim.

2) External Validity: PyBCV-420 covers six Python repositories and three LLMs. BCV rates for other repositories, languages, or future model versions may differ. Our finding of consistent rates across repositories (35.5–41.9%) provides partial evidence for generalisability, but replication on other corpora is warranted. VeriDoc is Python-specific; extension to TypeScript and Java is architecturally straightforward but not evaluated here.

3) Construct Validity: Runtime test execution provides strong evidence for RSV, PCV, SEV, and ECV violations, but COV violations (omissions) cannot be fully operationalised as executable tests. Our reported F1 for COV (0.57) reflects this fundamental limitation rather than an implementation deficiency. Researchers using PyBCV-420 should weight COV results accordingly.

4) Flaky Tests: 28% of false negatives are attributable to test synthesis failures that produce UNDETERMINED outcomes. These failures do not produce false positives (they are silently skipped), but they reduce recall. Future work should investigate sandboxed execution environments and multi-attempt synthesis strategies to reduce flakiness rates.

VIII. DISCUSSION

A. Implications of the 38.4% Prevalence Finding

The finding that 38.4% of LLM-generated docstrings contain at least one verifiably false behavioral claim has practical implications that extend beyond the detection task. In a project with 500 AI-documented functions, this rate implies approximately 192 functions with misleading documentation—an average of 1.4 false claims per violated function. Compounded across an organization-wide deployment of code generation tooling, the aggregate documentation error surface is substantial.

The consistency of this rate across all three LLMs (34.8–43.7%) argues against the interpretation that the problem is model-specific or amenable to simple prompt engineering fixes. The underlying cause is likely structural: language models estimate the most probable documentation given a function signature, without executing the code or performing formal verification. Correctness is not a training objective for documentation generation.

B. Relationship to Developer Trust

Adoption of commit-time verification tools is sensitive to false-positive rate. Our FPR of 0.19 means approximately one in five flags is a false alarm—a rate we consider acceptable for an initial deployment but which must continue to improve. The graduated reporting model (block on high-confidence, warn on lower-confidence) provides a tunable trade-off analogous to the threshold-configurable behavior of `mypy` in strict versus permissive modes. User studies on developer responses to violation reports are a priority for future work.

C. Future Directions

Improving COV detection is the highest-priority research direction, as it requires fundamentally different methods—likely combining static branch analysis with targeted LLM reasoning over execution path coverage. Multi-language extension (TypeScript, Java) is the highest-priority engineering direction. Finally, a longitudinal study measuring whether commit-time violation feedback influences how developers formulate prompts for subsequent code generation would address an open question about second-order effects of automated verification on generation quality.

IX. CONCLUSION

We have presented an empirical study and automated detection pipeline for behavioral contract violations in LLM-generated Python docstrings. Our core contributions are a six-category violation taxonomy grounded in systematic analysis, a three-stage detection pipeline that uses dynamic test synthesis to produce runtime-grounded verdicts, and the PyBCV-420 benchmark that enables reproducible evaluation.

The central empirical finding—that 38.4% of LLM-generated docstrings contain at least one verifiably false behavioral claim, consistently across three major providers—establishes a

baseline prevalence measurement for this class of defect and motivates both improved generation methods and continued development of detection and verification tooling. Our pipeline achieves F1 = 0.77, with false-positive rate 0.19, substantially outperforming LLM-as-judge and test-and-interpret baselines. The ablation result that supplying the function body to the test synthesizer degrades performance to 0.44 provides a concrete design principle for any future system in this space: tests must be derived from the claim, not from the code.

Acknowledgment

The authors thank the Department of Artificial Intelligence and Machine Learning, Rajalakshmi Engineering College, for computational resources and research support.

References

[1] GitHub, "GitHub Copilot: AI pair programmer," *GitHub Blog*, 2025. [Online]. Available: https://github.com/features/copilot

[2] Y. Liu *et al.*, "CodeHalu: Investigating code hallucinations in LLMs via execution-based verification," in *Proc. AAAI Conf. Artif. Intell.*, 2024.

[3] GitClear, "Coding on Copilot: 2025 data shows AI is degrading code quality," *GitClear Technical Report*, 2025. [Online]. Available: https://gitclear.com

[4] J. Spracklen *et al.*, "Slopsquatting: LLM package hallucinations as a supply-chain attack vector," in *Proc. USENIX Security Symp.*, 2025.

[5] R. Alrashedy and F. Palomba, "METAMON: Finding inconsistencies between program documentation and behavior using metamorphic testing," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reeng. (SANER)*, 2025.

[6] Z. Wang *et al.*, "CCIBench: A benchmark for code-comment inconsistency detection and repair," in *Proc. Int. Conf. Softw. Eng. (ICSE)*, 2025.

[7] F. Palomba, "Semcheck: Specification-driven code verification for pre-commit pipelines," *GitHub Repository*, 2025. [Online]. Available: https://github.com/semcheck/semcheck

[8] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[9] G. Sridhara, E. Hill, and L. Pollock, "Towards automatically generating summary comments for Java methods," in *Proc. IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2010.

[10] S. Panthaplackel *et al.*, "Associating natural language comment and source code entities," in *Proc. AAAI Conf. Artif. Intell.*, 2021.

[11] D. R. MacIver, D. Hatfield-Dodds, *et al.*, "Hypothesis: A new approach to property-based testing," *J. Open Source Softw.*, vol. 4, no. 43, 2019.

[12] S. Lukasczyk *et al.*, "Pynguin: Automated unit test generation for Python," in *Proc. ICSE Companion*, 2022.

[13] M. Schäfer *et al.*, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Softw. Eng.*, 2024.

[14] M. Honnibal *et al.*, "spaCy: Industrial-strength natural language processing in Python," *Zenodo*, 2020. doi:10.5281/zenodo.1212303.

[15] P. Zhou *et al.*, "Automatic detection of outdated API usage in software documentation," in *Proc. Int. Conf. Softw. Eng. (ICSE)*, 2022.

[16] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, 2020.